Meeting 17. Representations

Similarity · Image graphs · Hitting probabilities · Spectral coordinates · Genetic PCA

A representation records selected aspects of an object in a form we can work with. An image can be a grid of colors, a collection of points, or a weighted graph. Each view makes different relationships explicit. Moving between them can turn a visual question into a calculation, or make a pattern visible that was difficult to recognize in the original data.

Consider a plot in which each point represents a person and the two coordinates come from genetic measurements. The points resemble a map of Europe, although geographic coordinates were not inputs to the PCA calculation. Understanding this result requires several distinctions: the person and their measurements, the measurements and their numerical encoding, and the full collection of features and its two-dimensional summary. We will return to this example after developing the same ideas with images and graphs.

Read and explore.

The figures below are interactive. Change a setting, predict its effect, and compare the result with the explanation. Reset restores the stated example. The slide deck presents the same demonstrations. Assignment 3 develops the image and graph computations. The PDF preserves representative figure states and includes the expanded mathematical explanations.

The main points

1. **A representation should serve a question**

   Encoding, modeling, and summarizing are different operations. Ask what each transformation preserves, assumes, and discards. Section 1.

2. **Pixel features and graph edges describe different relationships**

   RGB features describe appearance. Neighborhood edges record spatial relations. Affinities combine these choices into a weighted graph. Section 2.

3. **Hitting probabilities connect walks to linear systems**

   Conditioning on the first step gives a weighted-average equation. Collecting these equations gives a Laplacian system with fixed seed values. Section 3.

4. **Useful graph coordinates vary smoothly across strong edges**

   Laplacian eigenvectors provide coordinates for graph drawings. The graph stays fixed while the drawing changes. Section 4.

5. **PCA coordinates summarize variation in numerical features**

   Allele counts give each person a feature vector. PCA scores give each person a point in a lower-dimensional view. Section 5.

1. Representations and similarity

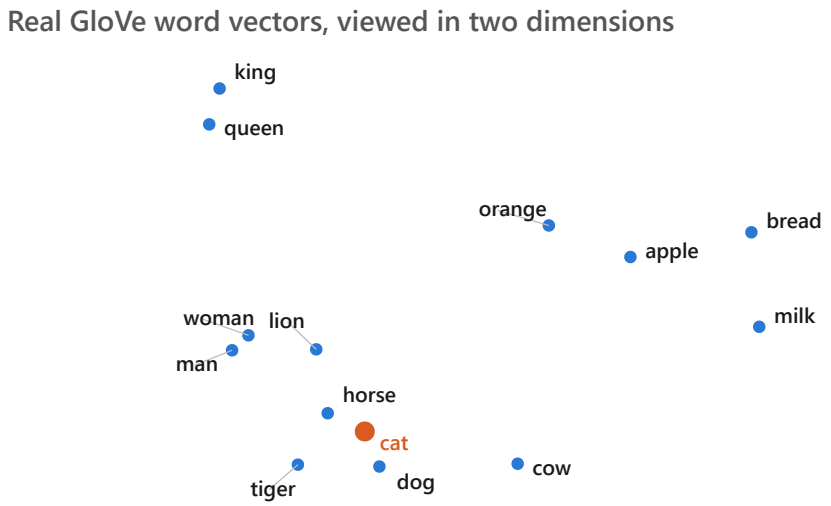
Objects, measurements, and encodings

A photograph already represents a scene: it records a two-dimensional projection, a finite set of pixels, and a limited range of color values. An RGB image of height H and width W can be stored as an $H \times W \times 3$ array. If we retain a fixed pixel order and the dimensions, reshaping it into an $N \times 3$ matrix, where $N = HW$, is reversible. If we keep only an unordered collection of color points and discard the positions, the spatial arrangement is no longer available.

It helps to distinguish three operations. **Re-encoding** changes how the same information is stored, as when a graph becomes an edge list or an adjacency matrix with a known node order. **Modeling** introduces choices about which relationships matter, as when we connect only neighboring pixels. **Summarizing** retains selected information, as when thousands of measurements become two coordinates. These operations can occur in the same workflow, but their consequences are different.

Language: structure and context

A grammar describes possible arrangements of words. A parse tree can make the grammatical roles in "the cat sleeps" explicit. A word embedding instead assigns a numerical vector to a word based on patterns in its context. That representation makes numerical comparisons and matrix computations possible. Grammars and vectors emphasize different relationships; they can coexist, and learned representations can contain information about syntax.



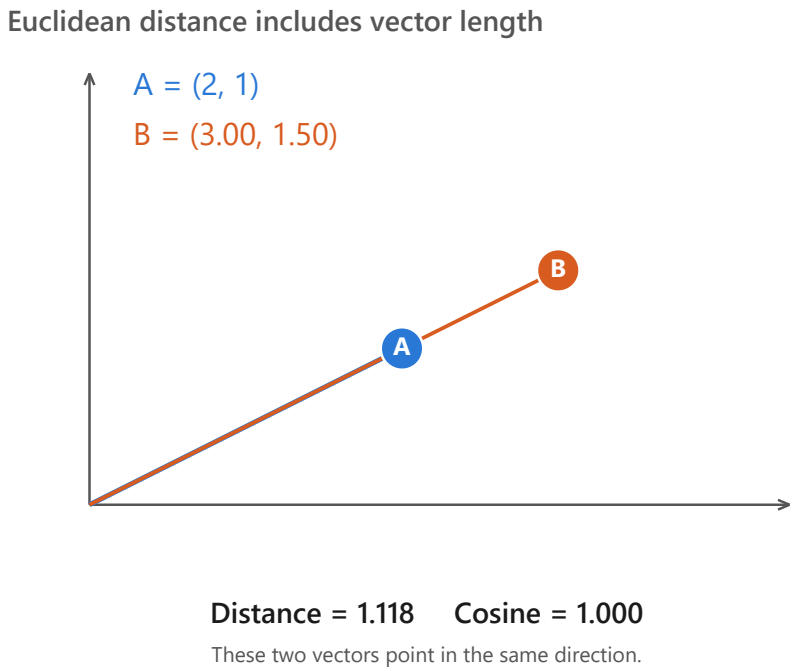
Fourteen real GloVe word vectors, projected into two dimensions for display. The similarity values use all 50 original coordinates. Similar contexts do not guarantee interchangeable meanings.

Choose what "similar" means

For two feature vectors x and y, Euclidean distance measures separation, while cosine similarity compares their directions:

$$\text{distance}\,(x,y) = \sqrt{\sum_k (x_k - y_k)^2}, \qquad \text{cosine}\,(x,y) = \frac{x^{\mathsf{T}} y}{\|x\|\,\|y\|}.$$

Cosine similarity requires nonzero vectors. Multiplying a vector by a positive scalar changes its length but not its direction, so its cosine similarity with another vector is unchanged. Euclidean distance generally changes. Neither measure is universally best: the meaning and scale of the features determine which comparison is useful.



Move B along the same ray as A. The vectors remain aligned while their Euclidean distance changes.

A two-dimensional display of word vectors adds another transformation. Distances in that display need not match similarities computed in the original space. The plot is a summary, not a replacement for all the original coordinates.

2. An image as a weighted graph

Specify the task before choosing the representation

Suppose we want to separate an object from the background of an image. We label a few pixels as **foreground** and a few as **background**. These labeled pixels are called **seeds**. The task is to assign the remaining pixels using their appearance, their spatial relationships, and the supplied labels. The seeds specify which object or region we want; the colors alone do not define "foreground."

A few labels specify the region we want

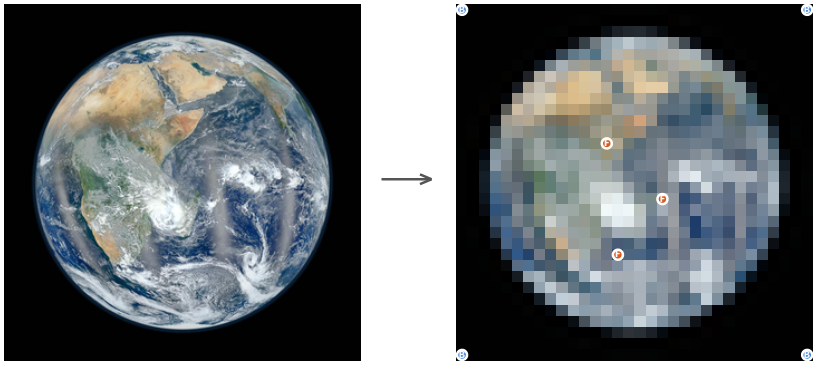


Image Image + seeds

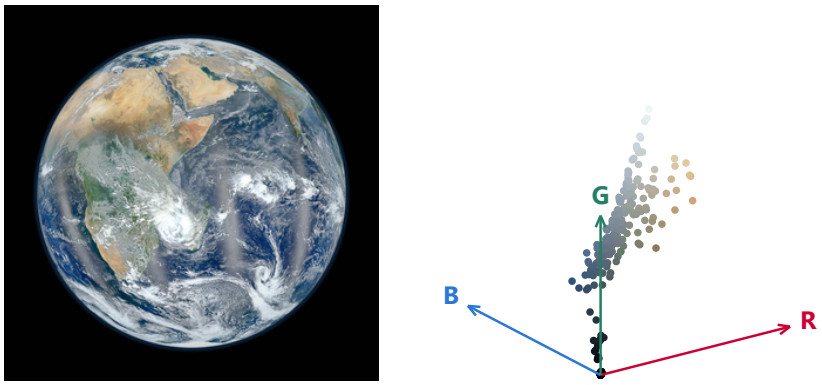
How should the labels extend to the other pixels?

Orange: foreground · blue: background

The same image before and after a few labels are supplied. Orange seeds specify foreground; blue seeds specify background.

Each pixel has an RGB feature vector $x_i = (r_i, g_i, b_i)$. Here the components are scaled to the interval $[0, 1]$. These triples become rows of $X \in \mathbb{R}^{N \times 3}$, with N equal to the number of pixels. Plotting the rows gives a cloud in color space.

The same pixels, in image space and RGB space



row, column red, green, blue

Which pixels are neighbors?

NASA/GSFC · Blue Marble 2012

Rotate the RGB cloud. A color point records appearance, but the cloud alone does not tell us which pixels touch in the image.

Two distant red objects can have nearly identical RGB values. To distinguish them, we need information that RGB similarity does not express. We therefore retain the spatial relationships between pixels.

Nodes, neighbors, and affinities

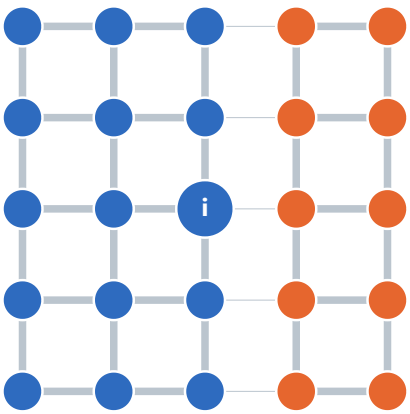
A **graph** consists of nodes and edges connecting pairs of nodes. Make each pixel a node. A four-neighbor graph connects pixels immediately above, below, left, and right; an eight-neighbor graph also connects diagonals. At image boundaries there are fewer valid neighbors. No edge wraps from the end of one row to the beginning of the next. Choosing the neighborhood is a modeling decision about which direct interactions matter.

An unweighted grid records where pixels are, but ignores their colors. To incorporate appearance, assign a nonnegative **affinity**, or similarity weight, to each neighboring pair:

$$w_{ij} = \exp\left(-\beta \left\| x_i - x_j \right\|^2 \right) \quad \text{for neighboring pixels;} \qquad w_{ij} = 0 \quad \text{otherwise.}$$

Identical colors have weight 1. Larger color differences produce smaller weights. At $\beta = 0$, every existing edge has weight 1, so color has no effect. Increasing β makes the model more sensitive to a given color difference. This is an illustrative RGB similarity; RGB distance is not a perceptually exact measure of color difference.

Color-sensitive edges · β = 20



Within-color weight: 1; cross-color weight: 1.0 × 10^{-6}

Positive floor 10^{-6} on existing edges in numerical calculations.

Change β and observe the edges across the color boundary. A stronger edge expresses greater affinity. The neighborhood is fixed while the weights change.

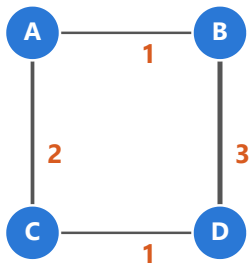
Numerical detail.

The demonstrations use $\max\{\exp(-\beta\|x_i - x_j\|^2), 10^{-6}\}$ on existing edges. This positive floor keeps the image graph connected. Non-neighbors still have weight zero. Very unequal weights can nevertheless make the linear system difficult to solve accurately.

The same graph in several storage forms

A drawing, an edge list, and a matrix can encode the same graph. We will use four nodes with edges $AB = 1$, $AC = 2$, $BD = 3$, and $CD = 1$. Fix the order (A, B, C, D). The affinity matrix W has entry w_{ij} in row i, column j. For this undirected graph, $W = W^{\mathsf{T}}$.

Same weighted graph; order A, B, C, D



	A	B	C	D
A	0	1	2	0
B	1	0	0	3
C	2	0	0	1
D	0	3	1	0

(A, B, 1) (A, C, 2)

(B, D, 3) (C, D, 1)

The node order connects each row to a node.

One graph, represented by a drawing, an edge list, and a matrix. The node order is essential when translating between them.

For an unweighted graph, a row of its adjacency matrix is a 0–1 vector recording which other nodes are neighbors. For a weighted graph, the entries are weights. Thus nodes can themselves be represented by feature vectors describing relationships. These adjacency rows are not drawing coordinates. Ordinary PCA of centered adjacency rows is a possible analysis, but it is different from the Laplacian layout developed below.

For a large image graph, each node has only a few neighbors. A sparse matrix stores the nonzero entries and their locations instead of allocating a value for every possible pair. It preserves the graph while changing how efficiently we can store and compute with it.

3. Walks, probabilities, and segmentation

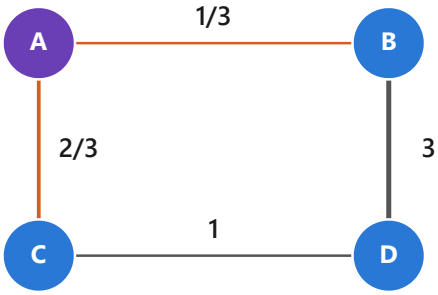
Weights become transition probabilities

A **weighted random walk** chooses its next neighbor in proportion to the edge weight. Let $d_i = \sum_j w_{ij}$ be the total weight incident to node i, also called its weighted degree. Then

$$P(i \to j) = \frac{w_{ij}}{d_i}.$$

The denominator includes all incident weights. At node A in our example, the weights to B and C are 1 and 2, so the transition probabilities are 1 / 3 and 2 / 3. A raw weight is not yet a probability; normalization makes the outgoing probabilities sum to one. Our example graphs have no isolated nodes, so every node has a positive denominator.

One fixed graph; normalize the outgoing weights



P(A → B) = 1/3; P(A → C) = 2/3

Every transition is to an existing neighbor.

AB = 1 · AC = 2 · BD = 3 · CD = 1

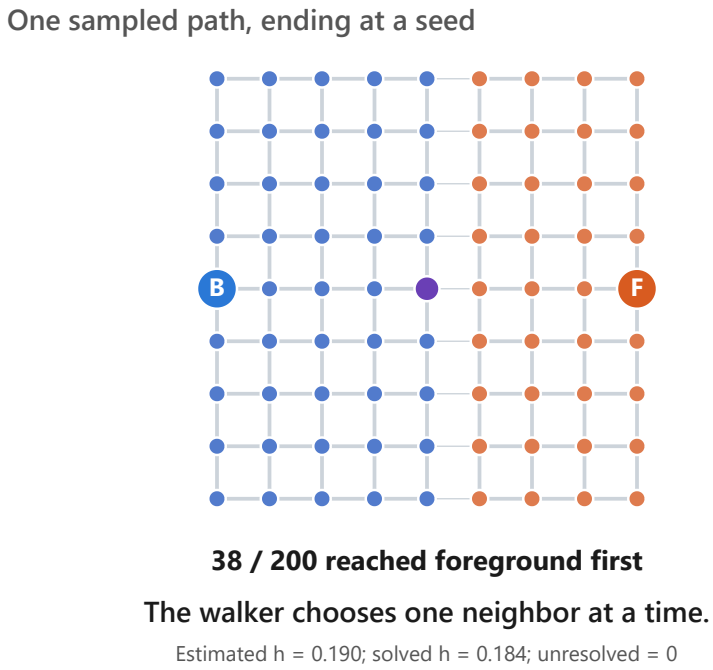
Select A or C and read its outgoing probabilities. The weights and graph stay fixed.

Ask which seed is reached first

On the image graph, strong affinities make similar neighboring pixels easier to move between. A walk can travel through a region using many local steps, so the question involves connectivity beyond the starting pixel's immediate neighborhood. Stop a walk when it first reaches any seed, and define

$$h_i = \Pr(\text{foreground seed is reached before any background seed} \mid \text{start at } i).$$

A foreground seed has $h_i = 1$; a background seed has $h_i = 0$. At an unseeded pixel, the value is unknown. This is a **first-arrival probability**, not a long-run visit frequency such as a PageRank score. If every connected component contains a seed, absorption occurs with probability one on these finite graphs. Otherwise some components have no seed to reach and their labels are not determined by this model.



Run one walk, then estimate with 200 walks. The observed foreground fraction is compared with the probability computed from a linear system. A simulation has sampling error; any paths that reach the step cap are reported separately.

The probability describes the specified graph and seeds. It is not automatically a calibrated probability that a pixel belongs to a real-world object. Affinities, connectivity, and seed choices can all be inappropriate for the intended image task.

The first-step equation

We can compute the hitting probabilities without sampling many paths. Condition on the next step: if a walk moves from i to j, its probability of subsequently reaching foreground first is h_j. Averaging over the possible next neighbors gives

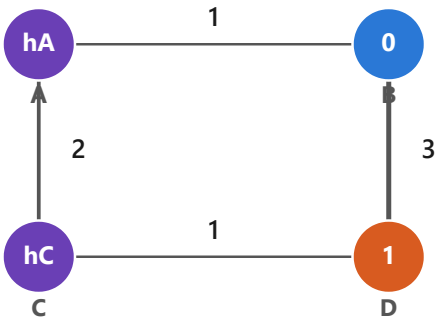
$$h_i = \sum_j P(i \to j) h_j = \frac{1}{d_i} \sum_j w_{ij} h_j \,.$$

Use our four-node graph, with B fixed as background and D as foreground. The unknowns are A and C. Their equations are

$$h_A = \frac{h_B + 2h_C}{3}, \qquad h_C = \frac{2h_A + h_D}{3}, \qquad h_B = 0, \quad h_D = 1 \,.$$

Neither unknown has to be found first. Both equations must hold simultaneously. The edge from B to D remains part of the graph, but does not affect these unknown probabilities: the walk stops as soon as it reaches either seed.

The first step determines a weighted average



$$hA = (1/3)hB + (2/3)hC$$

Two unknown probabilities; two simultaneous equations.

A walk stops at B (background) or D (foreground).

The same graph now carries probabilities. Known seed values are 0 and 1; the two other node values must satisfy the averaging equations together.

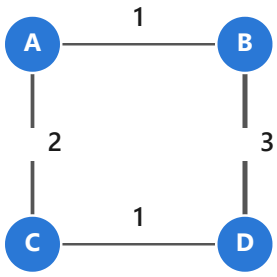
The Laplacian follows from the averaging rule

Multiply by d_i and move all terms to the left:

$$d_i h_i - \sum_j w_{ij} h_j = 0 .$$

Let D be the diagonal matrix with entries d_i. The **graph Laplacian** is $L = D - W$. Its diagonal entry gives a node's total incident weight, and each off-diagonal entry is the negative of the corresponding affinity. Thus one row of the matrix expresses one local averaging equation. Every row sums to zero.

The same weighted graph, written as local balance



L = D − W

	A	B	C	D
A	3	-1	-2	0
B	-1	4	0	-3
C	-2	0	3	-1
D	0	-3	-1	4

At A: 3hA − hB − 2hC = 0

The matrix comes from the averaging equation.

$(Lh)_i = 0$ at unseeded nodes; seed values stay fixed.

The Laplacian of the same four-node graph. The A row says 3hA − hB − 2hC = 0. This equation is imposed at unseeded nodes; seed values are fixed separately.

We do not solve $Lh = 0$ at every node and then hope to obtain a segmentation. Without the seed constraints, constant vectors already satisfy that equation. We impose the averaging equations only at unseeded nodes and retain the specified seed values.

Solve the unknown probabilities

Substituting $h_B = 0$ and $h_D = 1$ gives

$$\begin{bmatrix} 3 & -2 \\ -2 & 3 \end{bmatrix} \begin{bmatrix} h_A \\ h_C \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \qquad h_A = \frac{2}{5} = 0.4, \quad h_C = \frac{3}{5} = 0.6\,.$$

For example, $(0 + 2 \cdot 0.6) \,/\, 3 = 0.4$ verifies the averaging equation at A. Repeated walks from A would estimate 0.4; solving the system computes it directly, subject to numerical rounding.

The general system with fixed seeds

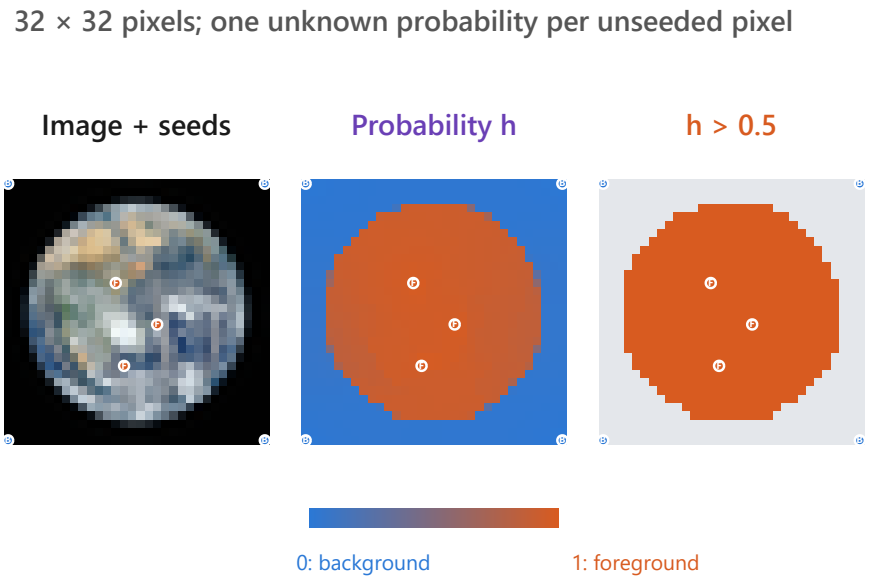
Partition the nodes into unseeded nodes U and seeds S. The rows for unseeded nodes give

$$L_{UU}h_U + L_{US}h_S = 0, \qquad L_{UU}h_U = -L_{US}h_S.$$

The seed values on the right are known. With nonnegative undirected weights and at least one seed in every connected component, the reduced matrix L_{UU} is positive definite, so the unknowns have a unique solution. Use a linear solver; explicitly forming an inverse is unnecessary.

Return the answer to image space

For an image, solve for one probability per unseeded pixel, then put those values back into their row and column locations. The result is a **probability map**. A simple decision rule labels a pixel foreground when $h_i > 0.5$, giving a binary **mask**. Thresholding discards the distinction between probabilities on the same side of the threshold; a mask alone does not show how close a decision was.

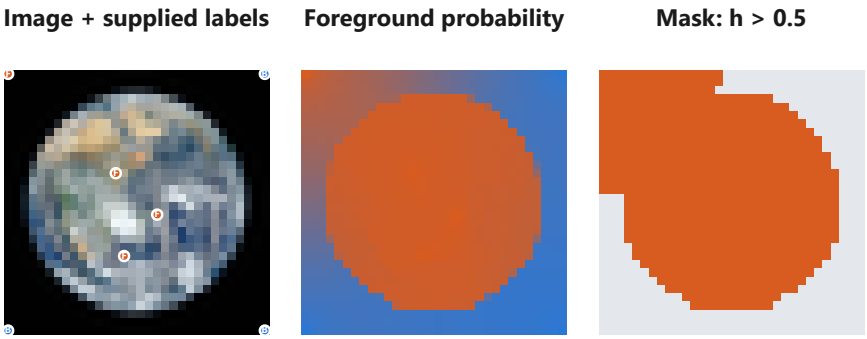


Click the left image to add or erase seeds. The middle panel shows foreground hitting probabilities; the right panel thresholds them. The 32 × 32 image uses four-neighbor affinities and an actual linear solve.

An incorrect seed.

In the meeting, deliberately assigning incorrect foreground and background labels changed the segmentation. The model must honor these labels exactly. A nearby walk is likely to hit the incorrect seed first, so one incorrect constraint can affect many pixels. This is propagation of an incorrect hard label; the experiment alone does not establish overfitting.

One background seed mislabeled as foreground



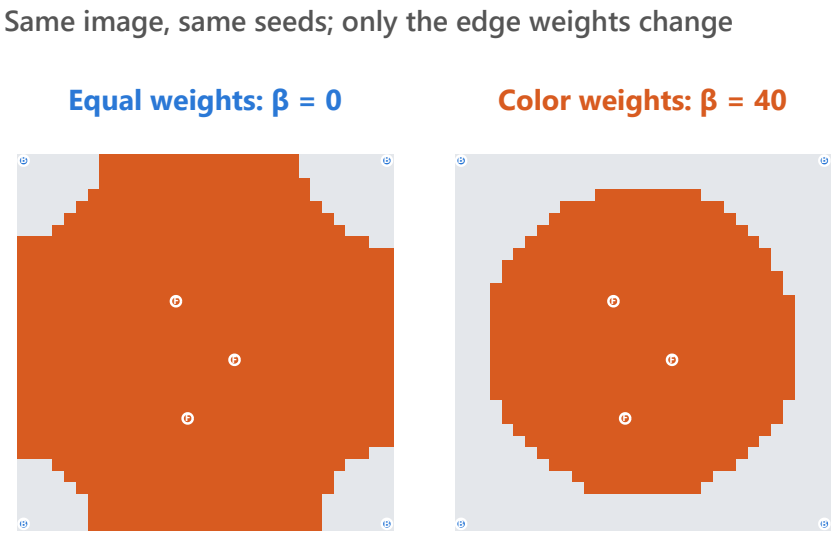
The specified label is enforced exactly.

An incorrect constraint can affect more than one pixel.

The probability is conditional on the graph and the supplied seeds.

Compare correct seeds with one or two incorrect seeds. The model cannot reject a seed value it is required to satisfy.

Changing affinities can also change the result. At $\beta = 0$, only grid connectivity and seeds influence the probabilities. Positive β makes color differences harder to cross. Increasing it is not a universal improvement: important boundaries may have similar colors, and pixels in the same object may look different. The output follows both the observations and the model.



Compare equal weights with color-sensitive weights while keeping the image, neighborhood, and seed labels fixed.

4. Graphs as coordinates

A graph and its drawing are different objects

A graph records connections. A drawing additionally assigns positions to nodes. Poorly chosen positions can hide a simple structure, even though the edges contain all the information needed to recognize it. Our next task is to choose two coordinates for each node so that the drawing reflects those relationships.

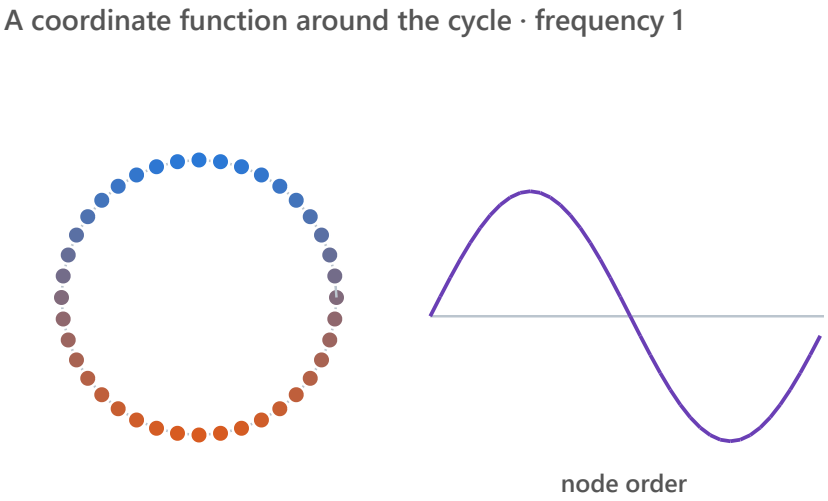
Segmentation assigned a scalar probability to each node. A drawing assigns two scalar coordinates, one for each axis. In both cases it is useful for strongly connected nodes to have similar values. The constraints differ: segmentation fixes some values at the seeds, whereas drawing must retain spread and provide independent coordinate directions.

Smooth values across strong edges

For a scalar value z_i at each node, define its graph energy

$$E(z) = \sum_{i<j} w_{ij}(z_i - z_j)^2 = z^{\mathsf{T}} L z .$$

The sum counts each undirected edge once. A large difference across a strong edge contributes a large penalty; a small difference or a weak edge contributes less. We call a coordinate **smooth on the graph** when its values change gradually across strong edges. Smoothness refers to the graph's connections, not to how the graph happened to be drawn initially.



Compare low and high frequencies around a cycle. Rapid changes across adjacent nodes have higher graph energy. The plotted node order is fixed, not sorted by the coordinate values.

A constant coordinate has zero energy, but it collapses every node to the same position on that axis. To obtain a useful first coordinate, require zero mean and unit length, then minimize energy. For a second coordinate, also require orthogonality to the first. These conditions remove the constant solution, fix scale, and prevent the second axis from repeating the first.

Why the same Laplacian appears in segmentation

Hold seed values fixed and minimize $E(h) = h^{\mathsf{T}} L h$ over the other values. Differentiating with respect to an unseeded value gives $2(Lh)_i = 0$, exactly the averaging equation. The seeded solution is a smooth extension of the specified labels. For graph coordinates we use the same energy, but replace seed constraints with spread and orthogonality constraints.

Eigenvectors supply the coordinates

An eigenvector q of L satisfies $Lq = \lambda q$. The scalar λ is its eigenvalue. A real symmetric Laplacian has an orthonormal eigenbasis. Since its energy is nonnegative for every z, it is **positive semidefinite**: its eigenvalues are nonnegative. This does not mean its entries are all positive.

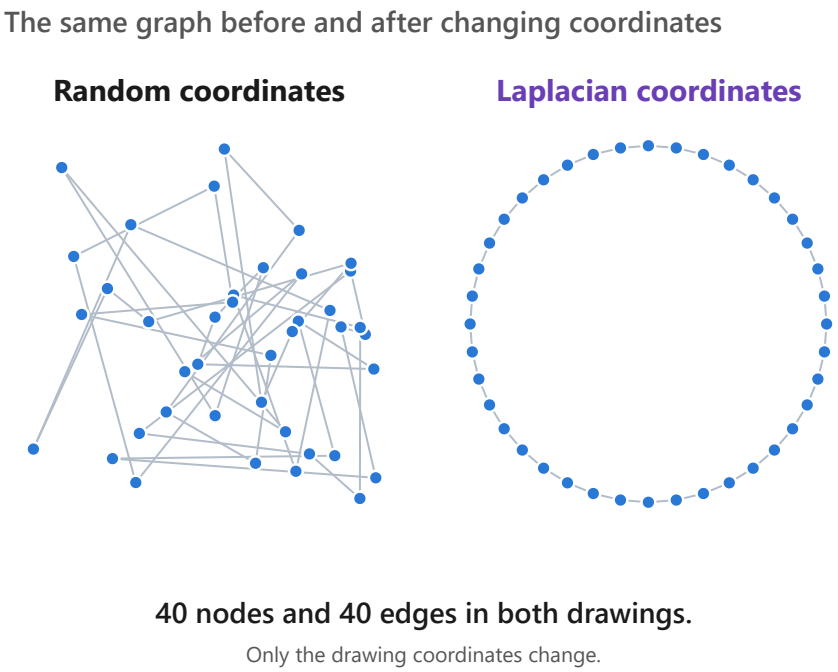
For a connected graph, order the eigenvalues as $0 = \lambda_1 < \lambda_2 \leq \lambda_3 \leq \cdots$. The first eigenvector is constant. Among zero-mean, unit-length coordinates, the second eigenvector minimizes energy. The third does the same with the additional orthogonality condition. Give node i the position

$$\text{position } (i) = \Big(q_2(i), q_3(i)\Big).$$

The coordinates come from the graph, not from the initial drawing. Eigenvector signs may flip, and repeated eigenvalues allow different orthonormal bases. Reflections or rotations of the resulting picture do not imply that its relationships changed.

> SVD, eigenvectors, and the distinction from PCA
>
> For this real symmetric positive semidefinite matrix,
>
> $$L = Q\Lambda Q^{\mathsf{T}}$$
>
> is also a valid SVD: take $U = V = Q$ and $\Sigma = \Lambda$, with consistent ordering. The equality is exact. Different numerical routines may choose signs or bases differently.
>
> Ordinary PCA applies SVD to a centered feature matrix and retains directions of largest variance. Here we decompose the Laplacian and use its smallest nonzero directions. The important differences are the matrix and the selected directions. Applying ordinary PCA to adjacency rows is not the same calculation.

The same graph becomes readable



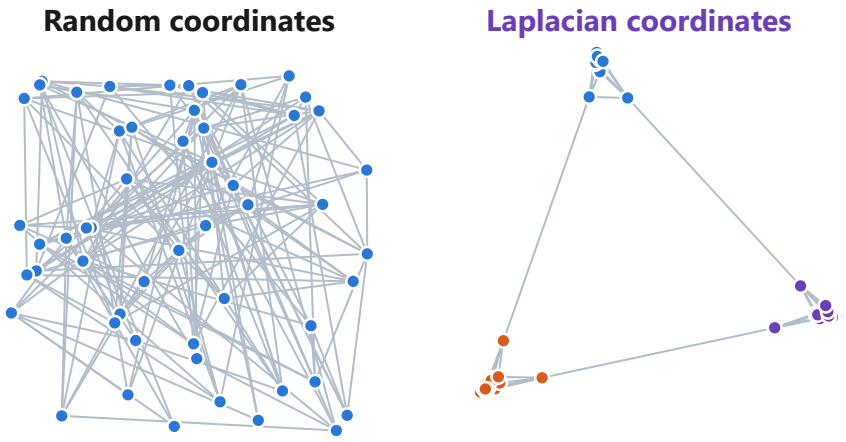
Move the slider from random coordinates to Laplacian coordinates. The 40 nodes and 40 edges never change. Two nonconstant cycle eigenvectors recover a circular drawing, up to rotation and reflection.

Intermediate slider settings interpolate between the initial and final drawing coordinates. They are not intermediate eigenvectors or a sequence of graph edits. For a general graph, a spectral layout does not guarantee a crossing-free or uniquely meaningful drawing. It gives a view designed to reflect the graph's relationships.

Community structure

A group with many internal connections and relatively few connections to the rest of the graph can appear as a concentrated region in spectral coordinates. Strong internal links discourage large coordinate differences within the group, while sparse links between groups allow their coordinates to differ more.

The same graph before and after changing coordinates

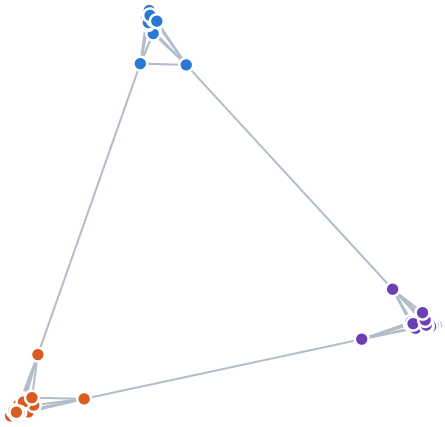


54 nodes; the same edges in both drawings.

The three-cluster coloring uses k = 3 supplied.

This is a synthetic graph with 54 nodes and three constructed dense groups. Move the layout slider, then toggle cluster colors. The coloring fits k-means to the coordinates with k = 3 supplied; it does not discover the number of groups.

0 additional cross-group edges



54 nodes · 160 edges

Each setting recomputes the embedding and a three-cluster partition.

Add cross-group edges and recompute the coordinates and three-cluster coloring. This control changes the graph itself. A forced partition still returns labels when the geometric separation becomes weak.

A visually separated group is evidence worth investigating, not a guarantee of a particular social or semantic category. Communities depend on the graph's construction, its weights, and the criterion used to identify groups.

NetworkX as a tool for changing representations

NetworkX provides a graph object, functions to convert it to numerical matrices, and drawing layouts. Keeping the same node order makes matrix rows correspond to the intended nodes. The following small graph is the one used in the probability calculation:

Graph, sparse matrices, and drawing positions

```
import networkx as nx

G = nx.Graph()
G.add_weighted_edges_from([
    ("A", "B", 1), ("A", "C", 2),
    ("B", "D", 3), ("C", "D", 1)
])
nodes = ["A", "B", "C", "D"]
W = nx.to_scipy_sparse_array(G, nodelist=nodes)
L = nx.laplacian_matrix(G, nodelist=nodes)
positions = nx.spectral_layout(G, weight="weight")
```

`W` stores affinities; `L` stores the Laplacian; `positions` maps each node to drawing coordinates. These objects answer different computational needs.

5. Genetic measurements as points

A numerical feature needs a meaning

Assigning arbitrary numbers to letters is not the encoding used in this example. At a selected autosomal variant site, a person has two inherited allele copies. An **allele** is one version at that site. For a site with possible alleles A and G, choose G as the allele to count. Then AA gives 0 copies, AG gives 1, and GG gives 2. The pair describes the two copies at the same site, not two adjacent letters in a DNA sequence.

Count the chosen allele at each selected site

Two copies at one site: AA → 0 AG → 1 GG → 2

	v1	v2	v3	v4	v5	v6	v7	v8	v9	v10
	2	0	1	1	0	0	2	1	2	2
	0	1	0	1	2	1	1	0	1	2
	2	0	1	0	1	1	1	0	2	1
people	0	1	0	2	1	0	1	1	1	0
	1	0	1	0	0	1	2	2	2	1
	1	1	2	0	1	0	2	1	1	1
	1	1	1	0	2	0	2	0	2	1
	2	1	2	1	1	1	2	0	2	1

10 selected variant sites in this toy table

A person becomes one row of numerical features.

The two letters are not adjacent letters in a DNA sequence.

A small simulated table illustrates the encoding. Each row is a person and each column is a selected site. G is the counted allele in the illustrative example; the chosen allele is defined separately at each marker.

With N people and p markers, the genotype-count matrix has shape $N \times p$. Each person is a point with p numerical features. The ten columns above are only a teaching example. The displayed published European analysis used 1,378 people and 32,991 markers.

PCA directions and person-coordinates

Center the marker columns and apply the chosen scaling. Let X now denote that centered and scaled matrix. With

$$X = U\Sigma V^{\mathsf{T}},$$

the columns of V are feature-space directions. The scores, which locate the people along those directions, are $XV = U\Sigma$. Retaining the first two gives

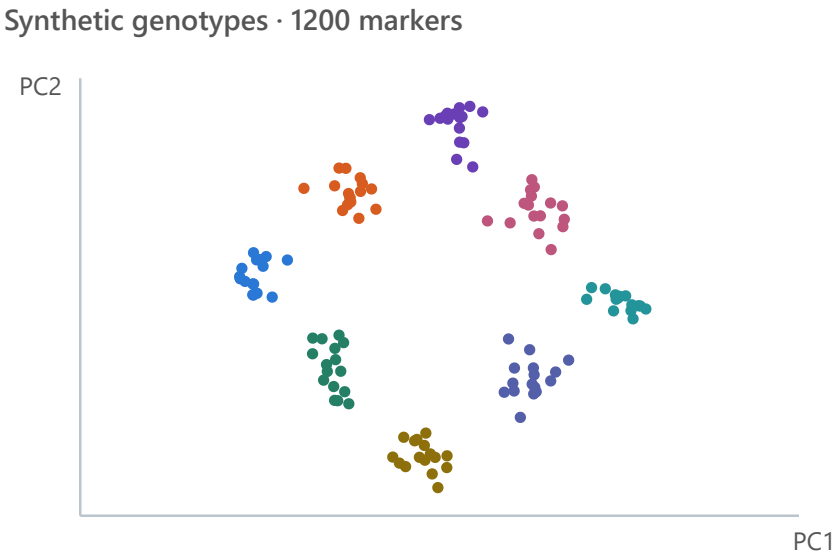
$$T = XV_2 = U_2\Sigma_2, \qquad T \in \mathbb{R}^{N \times 2}.$$

One row of T supplies two coordinates for one person: one point in the plot. For X of shape $N \times p$, $X^{\mathsf{T}}X$ is $p \times p$ and XX^{T} is $N \times N$. The feature covariance is $X^{\mathsf{T}}X \,/\, (N-1)$. SVD can obtain the directions and scores without explicitly constructing either large product.

Unlike the graph layout, PCA keeps the directions with the largest variance. Two coordinates summarize selected variation; they do not preserve every difference among individuals.

A controlled demonstration

The interactive example below simulates allele counts for 128 individuals from eight source regions. Allele frequencies vary with an underlying location in the simulation. PCA sees the genotype matrix, while the source-region labels are used only to color the resulting plot. This example illustrates a mechanism; it is not a new analysis of the real European data.



Two PCs retain 22.0% of this simulated variation.

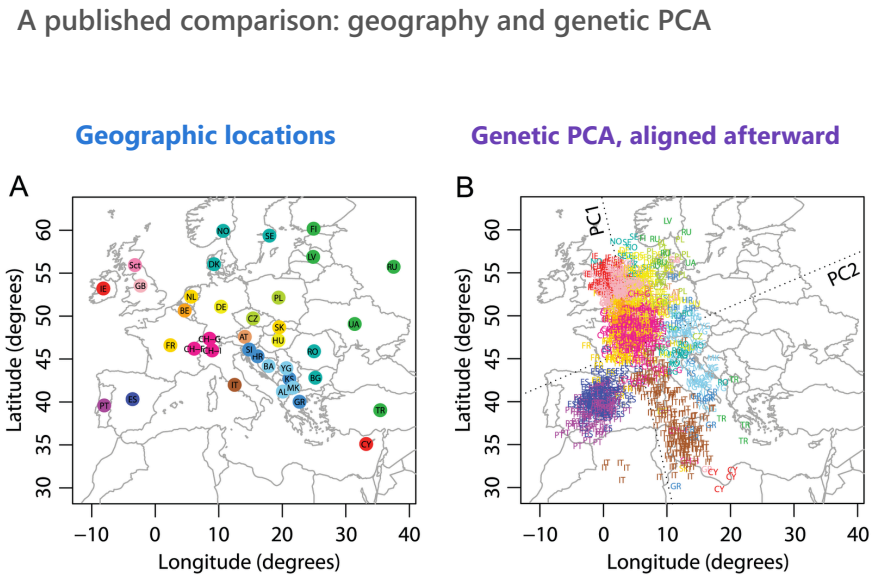
Colors were not used in PCA. Synthetic data; not the Europe study.

Change the number of simulated markers. Then shuffle individuals separately within each marker column. Each column retains its counts, but the correspondence across markers for an individual is broken.

Independent column shuffling is a useful comparison because it preserves each marker's histogram while disrupting the shared individual structure. More markers provide more measurements of the pattern built into this simulation; they do not guarantee a useful pattern for arbitrary real data.

> **Centering and scaling in this simulation**
>
> If the sample frequency of the counted allele at marker j is p_j, its mean count is $2p_j$. The example centers at that mean and scales by $\sqrt{2p_j(1-p_j)}$. Constant columns are removed. Real analyses also need choices about missing genotypes, quality control, correlated markers, and sampling. Those choices influence the representation supplied to PCA.

The published map of Europe



Wang et al. (2012), Fig. 2 · CC BY · published data, not a new replication

Published Figure 2 from Wang, Zöllner and Rosenberg (2012), CC BY. Genetic PCA coordinates are aligned with geography after PCA. This is the published figure, not a rerun of the individual-level data.

The original result of Novembre and colleagues (2008), and the follow-up shown here, demonstrate a correspondence between genetic variation and European geography. Genetic measurements supply the PCA calculation. Geographic labels and a subsequent alignment help interpret and compare the result. The alignment can rotate, reflect, translate, and uniformly scale the PCA configuration; it does not supply geographic coordinates as features to PCA.

In the displayed 2012 analysis, PC1 and PC2 explain about 0.30% and 0.16% of the total variation, respectively. A small fraction of total variation can still reveal a coherent geographic pattern. The other variation has not disappeared; it is omitted from this view. Sampling and preprocessing affect what appears.

The appropriate conclusion is that these genetic measurements contain geographic structure. The plot does not establish sharp biological borders or make nationality a quantity that PCA discovers. Country colors are annotations, and the two-dimensional map is one summary of the measured variation.

6. What each transformation enables

Transformation	What it enables	What to keep in mind
Image → RGB features	Compare pixel appearance numerically.	Retain pixel positions if spatial arrangement is needed.
Features + neighborhoods → weighted graph	Combine appearance with spatial relations.	Neighborhood and affinity are modeling choices.
Graph + seeds → hitting probabilities	Extend labels through the graph.	The result is conditional on the graph and fixed seed labels.
Graph → spectral coordinates	Reveal structure in a drawing.	The graph stays fixed; the drawing is a low-dimensional view.
Genotype counts → PCA scores	Display leading patterns of variation among people.	Two coordinates omit most measured variation.

A useful representation makes a question computable or a relationship visible. At each transformation, identify the object being represented, the meaning of its coordinates or entries, and the information that is preserved, assumed, or discarded. The same discipline applies to a grammar tree, a word vector, an image graph, and a genetic map.

Sources and data

1. Leo Grady (2006), Random Walks for Image Segmentation. The first-arrival formulation and its Laplacian system.
2. NetworkX spectral layout documentation. Coordinates from the nonconstant eigenvectors of the unnormalized Laplacian.
3. John Novembre and colleagues (2008), Genes mirror geography within Europe. The original geographic PCA result.
4. Chaolong Wang, Sebastian Zöllner and Noah A. Rosenberg (2012), A Quantitative Comparison of the Similarity between Genes and Geography in Worldwide Human Populations, Figure 2, CC BY. Source of the displayed European figure, sample size, marker count, and explained-variance values.
5. Jeffrey Pennington, Richard Socher and Christopher Manning (2014), GloVe. The word demonstration uses actual 50-dimensional course vectors; its two-dimensional view is a PCA projection.
6. NASA/GSFC, Blue Marble 2012. Image used in the pixel and segmentation demonstrations.

The cycle, community graphs, and genotype experiment are reproducible constructed examples. The community and genotype labels are not inputs to their spectral or PCA coordinate calculations. The real genetic figure is reproduced from the cited publication.